OFFICIAL TECHNICAL SPECIFICATION • V1.0.0

Sai Traders — Storefront REST API

External Integration Specification for Angular & Web Storefront Applications

PUBLIC CLOUD HOST	AUTH MECHANISM	TARGET GODOWN	ERP SYNC TARGETS
https://tunes-incidence-minutes-examples.trycloudflare.com	x-api-key Header	Sarees (Dedicated)	TallyPrime & Sheets

🔐 1. Authentication & API Key Credential

All endpoints under `/api/v1/store/*` require an authenticated API key. Requests without this header or with an invalid key will immediately return HTTP `401 Unauthorized`.

STOREFRONT API KEY (REQUIRED IN ALL HTTP REQUESTS)

```
sai_store_live_sec_892176
```

Pass this key in the `x-api-key` HTTP header (or as `Authorization: Bearer sai_store_live_sec_892176`).

🌐 Cross-Origin Resource Sharing (CORS) Policy

CORS is fully enabled with `Access-Control-Allow-Origin: *` and preflight `OPTIONS` support. Your Angular app running on `http://localhost:4200` or your live production domain can make direct HTTP requests without browser CORS errors.

📋 2. Storefront Endpoints Summary

Method	Endpoint Path	Headers	Purpose
`GET`	`/api/v1/store/products`	`x-api-key`	Fetch active catalog with photos, live inventory, and color variant trees.
`GET`	`/api/v1/store/products/:sku`	`x-api-key`	Lookup single saree or color group by base SKU (e.g. `SAI-VAR-01`).
`POST`	`/api/v1/store/orders`	`x-api-key` `Content-Type: app/json`	**Purchase Webhook:** Deducts stock, creates Tally Sales Voucher, and logs to Sheets.

🧪 3. Interactive Web Test Simulator

An interactive simulator testbed is deployed on the backend to preview catalog display, test color variant swatches, and simulate order webhooks live:

Live Web Simulator:
https://tunes-incidence-minutes-examples.trycloudflare.com/api-test
Includes live real-time API telemetry stream, one-click 401 test, and three-way sync checkmarks.

🥻 4. Catalog Retrieval: `GET /api/v1/store/products`

Returns published sarees ready for online storefront display. If an item has multiple colors (e.g., Maroon Red, Royal Blue, Emerald Green), it is marked with `"is_group": true` and contains a `variants` array with color swatches and individual variant stock.

Query Parameters

Parameter	Type	Default	Description
`in_stock_only`	Boolean	`false`	When set to `true`, only returns products currently in stock (> 0).
`search`	String	(none)	Case-insensitive search across saree name, fabric, or weave.

cURL Request Example

```
curl -X GET "https://tunes-incidence-minutes-examples.trycloudflare.com/api/v1/store/products?in_stock_only=true"
  -H "x-api-key: sai_store_live_sec_892176"
```

Catalog Response Schema (Truncated Sample)


```
{
  "success": true,
  "count": 9,
  "target_godown": "Sarees",
  "products": [
    {
      "id": "7923a3cc-9934-4453-85db-6ea914e1d690",
      "sku": "SAI-VAR-01",
      "name": "Royal Zari Heritage Silk Saree",
      "fabric": "Pure Kanjivaram Silk",
      "weave": "Traditional Zari",
      "price": 4800,
      "mrp": 6500,
      "stock": 13,
      "in_stock": true,
      "image_urls": ["https://.../saree-catalog-images/sai_var_01.jpg"],
      "is_group": true,
      "variants": [
        {
          "sku": "SAI-VAR-01-RED",
          "name": "Royal Zari Heritage Silk Saree (Maroon Red)",
          "color": "Maroon Red",
          "price": 4800,
          "stock": 6,
          "in_stock": true
        },
        {
          "sku": "SAI-VAR-01-BLU",
          "name": "Royal Zari Heritage Silk Saree (Royal Blue)",
          "color": "Royal Blue",
          "price": 4800,
          "stock": 4,
          "in_stock": true
        }
      ]
    }
  ]
}
```


🔍 5. Single SKU / Group Lookup: `GET /api/v1/store/products/:sku`

Fetch details for an individual saree product page. Supports querying by base color family SKU (e.g. `SAI-VAR-01`) or by exact variant SKU (e.g. `SAI-VAR-01-BLU`).

📦 6. Post-Purchase Webhook: `POST /api/v1/store/orders`

Called by your Angular website when a customer finishes payment. The backend processes the purchase atomically across three systems:

1. DATABASE INVENTORY
Decrements quantity in `saree_catalog` and `stock_pool`.

2. TALLYPRIME ERP
Posts Sales Voucher under active **Sarees** godown.

3. GOOGLE SHEETS
Appends transaction row to daily sales spreadsheet.

Request Headers

```
Content-Type: application/json
x-api-key: sai_store_live_sec_892176
```

cURL Webhook Request

```
curl -X POST "https://tunes-incidence-minutes-examples.trycloudflare.com/api/v1/store/orders" \
  -H "Content-Type: application/json" \
  -H "x-api-key: sai_store_live_sec_892176" \
  -d '{
    "order_id": "ANG-2026-9042",
    "customer": {
      "name": "Priya Sharma",
      "phone": "+919876543210",
      "email": "priya.sharma@example.com",
      "address": "Flat 402, Lotus Enclave, Indiranagar, Bengaluru, KA 560038"
    },
    "items": [
      {
        "sku": "SAI-VAR-01-BLU",
        "name": "Royal Zari Heritage Silk Saree (Royal Blue)",
        "quantity": 1,
        "price": 4800
      }
    ],
    "total_amount": 4800,
    "payment_status": "paid",
    "payment_method": "Razorpay / UPI",
    "notes": "Express delivery requested"
  }'
```

JSON Response Schema

```
{
  "success": true,
  "message": "Order [ANG-2026-9042] received! Inventory updated, Tally Sarees godown synced, and Google Sheets log
  "order_number": "ANG-2026-9042",
  "stock_updates": [
    { "sku": "SAI-VAR-01-BLU", "deducted": 1 }
  ],
  "tally_synced": true,
  "sheets_synced": true,
  "target_godown": "Sarees"
}
```

💻 7. Angular TypeScript Integration Snippet

Copy-paste ready Angular Service utilizing standard `HttpClient`:

```typescript
// src/app/services/saree-store.service.ts
import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class SareeStoreService {
  private baseUrl = 'https://tunes-incidence-minutes-examples.trycloudflare.com/api/v1/store';
  private apiKey = 'sai_store_live_sec_892176';

  constructor(private http: HttpClient) {}

  private getHeaders(): HttpHeaders {
    return new HttpHeaders({
      'Content-Type': 'application/json',
      'x-api-key': this.apiKey
    });
  }

  /** 1. Fetch Saree Catalog */
  getProducts(): Observable<any> {
    return this.http.get(`${this.baseUrl}/products`, { headers: this.getHeaders() });
  }

  /** 2. Fetch Single Saree or Color Group by SKU */
  getProductBySku(sku: string): Observable<any> {
    return this.http.get(`${this.baseUrl}/products/${encodeURIComponent(sku)}`, {
      headers: this.getHeaders()
    });
  }

  /** 3. Notify Backend of Completed Checkout (Triggers Tally + Google Sheets) */
  submitOrder(orderPayload: any): Observable<any> {
    return this.http.post(`${this.baseUrl}/orders`, orderPayload, {
      headers: this.getHeaders()
    });
  }
}
```

❓ 8. Error Codes & Troubleshooting

HTTP Status	Root Cause	Corrective Action
`401 Unauthorized`	Missing or wrong API key	Include `x-api-key: sai_store_live_sec_892176` in request headers.
`400 Bad Request`	Empty items array or missing SKU	Ensure `items` is a non-empty array with valid `sku` and `quantity`.
`404 Not Found`	SKU not found in catalog	Verify SKU exists against `GET /api/v1/store/products`.
`500 Server Error`	Internal exception	Check admin server log or open `/api-test` simulator to inspect stack trace.